C++ ile Bir ve Çok Boyutlu Diziler (Arrays)

Bellek Mimarisi, Algoritmalar

Bilgisayar Programlama

Fizik Bölümü

Ders Planı

1. Dizi Mimarisi ve Bellek Modeli
2. Çok Boyutlu Diziler (Satır Öncelikli Düzen)
3. Bilgisayar Uygulamaları
4. Kritik Notlar ve Sık Yapılan Hatalar

Dizi Nedir ve Neden Kullanılır?

- **Fiziksel İhtiyaç:** Skaler değişkenler (`double t, E;`) çok bileşenli durum vektörleri, alan dağılımları veya deneysel zaman serileri için yetersiz kalır.
- **Tanım:** Dizi (*array*), **aynı veri tipine** ait sabit sayıdaki elemanın bellekte (*RAM*) **ardışık (contiguous)** hücrelerde sıralandığı homojen veri yapısıdır.

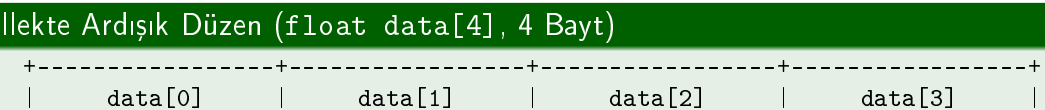
Temel Kurallar

- **Sıfır Tabanlı İndeksleme (*Zero-based Indexing*):** N elemanlı bir dizinin indisleri $[0, N-1]$ aralığındadır.
- **Doğrudan Erişim:** İndeks aritmetiği ile i-inci elemana anında ($O(1)$) erişilir:

$$\text{Adres}(A[i]) = \text{Taban Adres}(A) + i \times \text{sizeof(Veri Tipi)}$$

1B Dizilerin Bellek Yerleşimi

Bellekte Ardışık Düzen (`float data[4]`, 4 Bayt)



1D Dizi Tanımlama ve İlk Değer Atama:

```
const int N = 5;
double v[N];                          // Deger atanmamis (cop bellek degeri icerir)
double r[N] = {1.2, 3.4, 0.5, 4.8, 2.1};   // Ilk deger atamali liste
double sifirlar[N] = {0.0};           // Tum elemanlar 0.0 ile baslatilir
```

Çok Boyutlu Diziler: Satır Öncelikli Bellek Modeli

- Fizikte uzaysal alanlar (ızgaralar), tensörler ve operatörler çok boyutlu dizilerle ifade edilir:

$$M \in \mathbb{R}^{m \times n} \implies \texttt{double M[m][n];}$$

- Bilgisayar belleği fiziksel olarak tek boyutludur. C/C++ mimarisinde matrisler belleğe **Satır Öncelikli (Row-Major Order)** yazılır:

Adres Hesaplama Bağıntısı

$$\text{Adres}(M[i][j]) = \text{Taban Adres} + (i \times \text{Sütun Sayısı} + j) \times \text{sizeof(Veri Tipi)}$$

Önbellek (*Cache*) Performansı Uyarısı

Veri satır satır dizildiği için döngülerde **dış indisi satır (i), iç indisi sütun (j)** seçmek önbellek isabetini (*cache hit*) artırarak hesaplama süresini ciddi ölçüde hızlandırır.

Uygulama 1: Laboratuvar Ölçüm ve Hata Analizi

Fiziksel Problem: Bir deneyde elde edilen N adet bağımsız ölçümün $\{x_0, x_1, \ldots, x_{N-1}\}$ istatistiksel parametrelerinin hesaplanması:

Aritmetik Ortalama	Standart Sapma	Ortalamanın Hatası
$\bar{x} = \frac{1}{N}\sum_{i=0}^{N-1} x_i$	$s = \sqrt{\frac{\sum(x_i - \bar{x})^2}{N-1}}$	$\sigma_m = \frac{s}{\sqrt{N}}$

Algoritmik Adımlar:

1. Ölçümleri `double T[N]` dizisine yükle.
2. 1. Döngü: Toplam al ve $\bar{x}$ değerini bul.
3. 2. Döngü: Ortalamadan farkların karelerini biriktir; s ve σ_m değerlerini hesapla.

Uygulama 1: Kaynak Kod

```cpp
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;
int main() {
  const int N = 8;
  double T[N] = {2.05, 2.02, 2.08, 2.04, 2.03, 2.07, 2.01, 2.06};

  // 1. Ortalama Hesabi
  double toplam = 0.0;
  for (int i = 0; i < N; ++i) toplam += T[i];
  double ortalama = toplam / N;

  // 2. Standart Sapma ve Hata Hesabi
  double kareler_toplami = 0.0;
  for (int i = 0; i < N; ++i) {
    double fark = T[i] - ortalama;
    kareler_toplami += fark * fark;
  }
  double standart_sapma = sqrt(kareler_toplami / (N - 1));
  double standart_hata = standart_sapma / sqrt(N);

  cout << fixed << setprecision(4);
  cout << "Ortalama: " << ortalama << " s | Standart Hata: " << standart_hata << " s\n";
  cout << "Nihai Sonuc: (" << ortalama << " +/- " << standart_hata << ") s\n";
  return 0;
}
```

Uygulama 2: Koordinat Rotasyonu ve Matris Çarpımı

Fiziksel Problem: Klasik mekanikte bir konum vektörünün $\vec{r} = (x, y, z)^T$, z-ekseni etrafında θ kadar döndürülmesi ortogonal rotasyon matrisi ile ifade edilir:

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \implies r_i' = \sum_{j=0}^{2} R_{ij} r_j$$

İzometri / Uzay Metriği Korunumu

Rotasyon dönüşümü altında vektörün normu (boyu) korunmalıdır:

$$\|\vec{r}\| = \|\vec{r}'\| \implies \sqrt{x^2 + y^2 + z^2} = \sqrt{x'^2 + y'^2 + z'^2}$$

Uygulama 2: Kaynak Kod

```cpp
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;
int main() {
  const int BOYUT = 3;
  const double PI = 3.14159265358979323846;
  double theta = 45.0 * (PI / 180.0); // 45 derece radyana donusturuldu

  double R[BOYUT][BOYUT] = {
    {  cos(theta), -sin(theta), 0.0 },
    {  sin(theta),  cos(theta), 0.0 },
    {  0.0,              0.0,              1.0 }
  };
  double r[BOYUT] = {1.0, 1.0, 2.0};
  double r_prime[BOYUT] = {0.0, 0.0, 0.0};

  // Matris-Vektor Carpimi: r' = R * r
  for (int i = 0; i < BOYUT; ++i)
  for (int j = 0; j < BOYUT; ++j)
  r_prime[i] += R[i][j] * r[j];

  double norm_eski = sqrt(r[0]*r[0] + r[1]*r[1] + r[2]*r[2]);
  double norm_yeni = sqrt(r_prime[0]*r_prime[0] + r_prime[1]*r_prime[1] + r_prime[2]*r_prime[2]);

  cout << fixed << setprecision(4);
  cout << "|r| = " << norm_eski << " --> |r'| = " << norm_yeni << " (Norm Korundu)\n";
  return 0;
}
```

Uygulama 3: Sonlu Farklar ile Laplace Denklemi (2B Izgara)

Problem: Yüksüz ortamda elektrostatik potansiyel:

$$\nabla^2 V = \frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = 0$$

Merkezi Sonlu Farklar: $\Delta x = \Delta y$ olan kare ızgarada her iç düğüm noktası 4 komşusunun ortalamasıdır:

$$V_{i,j}^{(k+1)} = \frac{1}{4}\left(V_{i+1,j}^{(k)} + V_{i-1,j}^{(k)} + V_{i,j+1}^{(k)} + V_{i,j-1}^{(k)}\right)$$

Dirichlet Sınır Koşulları

- Üst kenar: $V = 100$ V
- Sol, Sağ, Alt kenarlar: $V = 0$ V (Topraklanmış)

Jacobi İterasyonu Mantığı

Güncelleme esnasında eski değerlerin bozulmaması için iki ayrı 2B ızgara kullanılır.

Uygulama 3: Kaynak Kod

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
  const int BOYUT = 6, ITERASYON = 100;
  double V[BOYUT][BOYUT] = {0.0}, V_yeni[BOYUT][BOYUT] = {0.0};

  // Dirichlet Sinir Kosullari
  for (int j = 0; j < BOYUT; ++j) { V[0][j] = 100.0; V_yeni[0][j] = 100.0; }

  // Jacobi Gevseitme (Relaxation) Dongusu
  for (int adim = 0; adim < ITERASYON; ++adim) {
    for (int i = 1; i < BOYUT - 1; ++i)
    for (int j = 1; j < BOYUT - 1; ++j)
    V_yeni[i][j] = 0.25 * (V[i+1][j] + V[i-1][j] + V[i][j+1] + V[i][j-1]);

    for (int i = 1; i < BOYUT - 1; ++i)
    for (int j = 1; j < BOYUT - 1; ++j)
    V[i][j] = V_yeni[i][j];
  }

  cout << "100 Iterasyon Sonrasi Potansiyel (Volt):\n" << fixed <<
  setprecision(1);
  for (int i = 0; i < BOYUT; ++i) {
    for (int j = 0; j < BOYUT; ++j) cout << setw(6) << V[i][j] << " ";
    cout << "\n";
  }
  return 0;
}
```

Kritik Notlar ve Sık Yapılan Programlama Hataları

Hata Türü	Fiziksel / Sayısal Etkisi	Doğru Yaklaşım
Sınır Aşımı (*Out of Bounds*)	$A[N]$ indeksine erişim tanımsız davranışa (*Segmentation fault*) yol açar.	Sayaçlar daima $0 \leq i < N$ aralığında sınırlandırılmalıdır.
Statik Boyut Kısıtı	Standart diziler çalışma anında büyütülemez; bellek taşmalarına neden olur.	Sabitler `const int` olmalı; dinamik veri için `vector` seçilmelidir.
Fonksiyona Aktarım (*Decay*)	Boyut bilgisi kaybolur, yalnızca taban adresi (*pointer*) iletilir.	Eleman sayısı fonksiyona ayrı bir parametre olarak eklenmelidir.
İlk Değer Atamama	Yerel diziler çöp bellek değeri içerir; toplam/fizik integrallerini bozar.	Akümülatör öncesi dizi açıkça sıfırlanmalıdır: `double A[N] = {0.0};`

Özet ve Sonuç

- **Doğrudan Erişim:** Diziler ardışık bellek yapısı sayesinde $O(1)$ sürede doğrudan bellek erişimi sunar.
- **Satır Öncelikli Düzen:** Matris işlemlerinde iç döngünün sütun indisi (j) olması önbellek verimini maksimize eder.
- **Sayısal Fizik:** Dizi yapıları istatistiksel veri analizinden çok boyutlu diferansiyel denklem çözümlerine (PDE) kadar tüm sayısal hesaplamaların omurgasını oluşturur.

Sorular?